

MIT OLL → QAI Enterprise Library

Course Extraction Process & Working Instructions

Standard operating guidance for all seven MIT courses

1. Purpose

Use this process to preserve the original MIT Open Learning Library course structure while systematically extracting educational, mathematical, algorithmic, experimental and engineering knowledge for the QAI Enterprise Library.

The process supports two goals: retain each MIT course as a usable solved-course knowledge set, and extract reusable patterns and primitive candidates without prematurely promoting educational material into production architecture.

2. Core Principle

Preserve course fidelity; perform QAI extraction as a separate layer.

Evidence first; pattern second; primitive third; implementation last.

Do not invent missing MIT course structure, implementation maturity, solutions, or QAI capabilities.

3. Standard Evidence Flow

- MIT OLL Course
- Original Course Hierarchy
- Subunit / Module
- Lecture / Content / Problem Set
- Question
- Solved Question
- Notebook / Experiment / Implementation
- Algorithm / Mathematical Model
- Recurring Pattern
- Primitive Candidate
- Validation
- QAI Enterprise Library

4. Three-Level Documentation Model

4.1 Course Level

Keep the course-level document compact: course identity, MIT hierarchy, subunits, major topics/algorithms, extraction status, major patterns, primitive candidates, cross-course relationships and final summary.

MIT OLL → Evidence → Pattern → Primitive → Validation → QAI Enterprise Library

4.2 Module / Evidence-Package Level

Use the full standard course extraction template for a substantial module/evidence package: lectures, problem sets, solved questions, notebooks, algorithms/models, patterns, primitive candidates, QAI mapping, validation and provenance.

4.3 Question / Notebook Level

Use Markdown and original source files for individual questions, solved questions, notebook summaries, experiments, patterns and implementation evidence.

5. Recommended Repository Structure

```
mit_courses/  
└─ courses/  
  └─ <course_id>/  
    ├── course_overview.md  
    ├── course_map.md  
    ├── curriculum.md  
    ├── extraction_status.md  
    ├── modules/  
    │   └─ <subunit>/  
    │       ├── README.md  
    │       ├── module_extraction.docx  
    │       ├── lectures/  
    │       │   └─ problem_sets/  
    │       ├── problem_sets/  
    │       │   ├── inventory.md  
    │       │   ├── source_files/  
    │       │   ├── solved_questions/  
    │       │   └─ extracted/  
    │       ├── notebooks/  
    │       │   ├── inventory.md  
    │       │   ├── raw/  
    │       │   ├── solved/  
    │       │   ├── reference/  
    │       │   └─ extracted/  
    │       ├── experiments/  
    │       ├── extracted_patterns/  
    │       └─ primitive_candidates/
```

6. Preserve the MIT Course Hierarchy

- Course → Subunit → Lecture / Problem Set / Assessment → Questions
- Preserve MIT item names and terminology.
- Capture question counts where provided.
- Retain MIT source URLs and provenance.
- Do not invent uninspected SU/module/lecture/problem-set names.
- QAI interpretation belongs in extraction/mapping layers.

MIT OLL → Evidence → Pattern → Primitive → Validation → QAI Enterprise Library

7. Problem Sets and Solved Questions

Solved questions are first-class evidence and remain associated with their original problem statements.

- Preserve the original problem statement.
- Capture each question separately where practical.
- Capture the solved question separately.
- Record reasoning, mathematical derivation, algorithm/procedure, assumptions and key insight.
- Record implementation clues and validation/results.
- Link notebooks or code to the corresponding question.
- Extract patterns only after reviewing problem and solution evidence.

8. Notebook Handling

- Preserve original .ipynb files.
- Maintain a notebook inventory.
- Record language, framework/SDK, dependencies and execution environment.
- Record input, output, algorithm/model and result.
- Create an extracted Markdown summary.
- Identify reusable implementation patterns.
- Do not claim production maturity from a solved notebook.

9. Pattern Extraction

- Name the pattern.
- Identify supporting source problems.
- Describe the common structure.
- Record inputs and outputs.
- Record control/feedback where relevant.
- Record evidence strength and limitations.
- Look for recurrence across modules and later across courses.

10. Primitive Candidate Extraction

- Identify the reusable capability.
- Separate the capability from the original application.
- Define proposed inputs and outputs.
- Describe API concept and runtime requirements.
- Identify dependencies.
- Define validation required.
- Assign maturity.
- Recommend promotion only when evidence supports it.

11. Example: HSP / Shor → AQSE

The supplied 8.371.3x example demonstrates the intended transformation.

- Source problem: quantum factoring / hidden-order context.

- Underlying mechanism: modular quantum dynamics expose spectral information.
- Probe: controlled operator / phase kickback.
- Observation: interference and measurement.
- Estimation: hidden eigenphase / spectral parameter.
- Feedback: choose or refine the next experiment.
- Reusable pattern: Probe → Measure → Estimate → Adapt.
- Primitive candidate: Adaptive Quantum Spectral Estimation (AQSE).
- Factoring is an application consuming the reusable estimation capability.

12. QAI Mapping Rules

- QAI Language
- QAI Primitives
- QAI OS
- QAI Runtime
- Hybrid Runtime
- QAI Control Plane
- Quantum Control Plane
- Adaptive Network Fabric
- Resource Registry
- Capability Registry
- QAI Labs
- QAI Product Foundry

13. Evidence Classification

- Educational knowledge
- Research
- Mathematical model
- Algorithm
- Pattern
- Prototype
- Implementation evidence
- Reusable capability
- Future capability

Never infer production maturity solely from educational or conceptual material.

14. Source / Format Policy

- DOCX: detailed module/evidence-package reports, solved-course packages and formal research/design documents.
- Markdown: GitHub canonical records, inventories, question summaries, notebook summaries, matrices, patterns and Enterprise Library entries.
- IPYNB: executable notebook evidence.
- Original PDFs/DOCX/problem sets: preserve as source material where appropriate.

15. Avoid Duplication

Do not copy a full primitive specification into every course where the same primitive appears. The course record should identify the candidate and link to the single Enterprise Library source of truth.

Course evidence → Primitive candidate → QAI Enterprise Library canonical entry

16. Cross-Course Convergence

- Same primitive appearing in multiple courses.
- Same mathematical model appearing in different contexts.
- Same feedback/control pattern.
- Same runtime requirement.
- Same QAI architecture capability.
- Complementary capabilities that should compose into one enterprise service.

17. Validation and Promotion

Lifecycle: Candidate → Pattern → Primitive Specification → Reference Implementation → Validation → Reusable Primitive → Enterprise Library

- Validate against additional problems.
- Prefer independent examples.
- Use simulator validation where appropriate.
- Use hardware/QPU validation where appropriate.
- Compare across courses.
- Document acceptance criteria.
- Do not promote a primitive based on one educational example alone.

18. Working Instructions for Another Session

- Provide the course ID and actual MIT OLL hierarchy being processed.
- Provide relevant problem sets, solved questions, notebooks or design documents.
- State whether the task is course mapping, module extraction, problem extraction, pattern extraction or primitive extraction.
- Use the standard template for the appropriate module/evidence package; do not create a huge monolithic course document.
- Preserve original MIT terminology and organization.
- Populate only fields supported by supplied evidence.
- Mark unavailable/unverified information as pending or not supplied.
- Keep QAI interpretation separate from MIT source evidence.
- Link recurring patterns to the canonical Enterprise Library entry rather than duplicating specifications.
- At completion, update extraction status and identify the next evidence package.

19. Completion Criteria

Area	Required Result	Status
Course hierarchy	Faithfully represented	Pending
Problem sets	Inventoried	Pending

MIT OLL → Evidence → Pattern → Primitive → Validation → QAI Enterprise Library

Solved questions	Captured where available	Pending
Notebooks	Inventoried and summarized	Pending
Algorithms/models	Evidence-based extraction	Pending
Patterns	Recurring patterns identified	Pending
Primitive candidates	Separated from production claims	Pending
QAI mapping	Architecture relationships recorded	Pending
Cross-course mapping	Performed after individual extraction	Pending
Validation	Defined for important candidates	Pending
Final summary	Course-level summary updated	Pending

20. Master Working Rule

PRESERVE → EXTRACT → ANALYZE → GENERALIZE → VALIDATE → MAP → PROMOTE

The objective is not simply to archive the MIT courses. It is to preserve high-quality educational evidence while systematically converting validated recurring knowledge into reusable QAI primitives, algorithms, runtimes, control capabilities and Enterprise Library assets.